

JIRA TASK 05

Build Basic Java Microservices System

Project Documentation

Task	Jira Task 05 — Build Basic Java Microservices System
Estimated effort	8 Hours
Technology	Java, Spring Boot, REST APIs, JPA/Hibernate, Maven, MySQL, Postman, Git/GitHub
Services	User Service, Product Service, Order Service
Repository	Microservice-architecture

Purpose: This document records the implementation approach, service structure, API testing process, architecture, Git/GitHub delivery, and troubleshooting notes for the task.

Prepared from the task requirements and project screenshots supplied for this task.

1. Task Objective

The objective of Jira Task 05 is to combine the microservices concepts learned in Epic 01 into a working Java Spring Boot system. The expected result is a working project demonstrating multiple services, separate data ownership, REST APIs, service-to-service communication, validation, error handling, testing, architecture documentation, README documentation, and Git delivery.

Required implementation steps

Step	Requirement
1	Create Spring Boot projects.
2	Create User Service.
3	Create Order Service.
4	Configure databases.
5	Create entities.
6	Create repositories.
7	Create services.
8	Create REST controllers.
9	Implement inter-service communication.
10	Add validation.
11	Add exception handling.
12	Test all APIs using Postman.
13	Create architecture diagram.
14	Write README.
15	Push to Git.

Final deliverable expected

- A working Java Spring Boot microservices project.
- Multiple independently structured services.
- Separate database/data ownership per service.
- REST APIs for the business operations.
- Service-to-service communication.
- Validation and error handling.
- Postman API test evidence.
- Architecture diagram and README.
- Source code pushed to Git/GitHub.

2. System Architecture

The task shows the main request flow as API Client → User Service → Order Service → Database. The implemented project also contains a Product Service, so the complete project can be represented as follows:

Component	Responsibility	Communication
API Client / Postman	Sends HTTP requests and verifies responses.	HTTP/REST
User Service	Manages user data and user-related REST endpoints.	HTTP/REST
Product Service	Manages product data, quantity and price.	HTTP/REST
Order Service	Handles order-related operations and communicates with another service when required.	HTTP/REST
Database	Persists service-owned data.	JPA/Hibernate + JDBC
Git/GitHub	Stores and shares the source code.	Git

Logical request flow

Client (Postman)
↓
User Service
↓
Order Service
↓
Service-owned Database

Additional product flow

Postman → Product Service → Product Database. Product operations are kept in the Product Service rather than mixing them into the User Service.

Microservices principle demonstrated

- Each service is a separate Spring Boot application.
- Each service has its own controller/service/repository/entity layers.
- Services communicate through HTTP rather than direct Java method calls.
- Each service should own its database/schema and persistence model.

3. Project Structure

The supplied screenshots show the workspace containing separate service folders. A clean structure for the project is:

```
microservices/
├── user-service/
│   ├── src/main/java/user_service/
│   │   ├── controller/UserController.java
│   │   ├── entity/User.java
│   │   ├── repository/UserRepository.java
│   │   └── UserServiceApplication.java
│   ├── src/main/resources/application.properties
│   └── product-service/
│       ├── src/main/java/product_service/
│       │   ├── controller/ProductController.java
│       │   ├── entity/Product.java
│       │   ├── repository/ProductRepository.java
│       │   └── ProductServiceApplication.java
│       ├── src/main/resources/application.properties
│       └── order-service/
│           ├── src/main/java/order_service/
│           │   ├── controller/OrderController.java
│           │   ├── service/OrderService.java
│           │   ├── repository/OrderRepository.java
│           │   └── OrderServiceApplication.java
│           ├── src/main/resources/application.properties
│           └── README.md
```

Layer responsibilities

Layer	Purpose
Entity	Represents database data using JPA annotations such as @Entity and @Id.
Repository	Provides database access through Spring Data JPA.
Service	Contains business logic and coordinates repository/client operations.
Controller	Exposes HTTP endpoints using Spring MVC annotations.
application.properties	Stores server port and database/application configuration.

The supplied User.java screenshot visibly shows a JPA entity with an ID generated using GenerationType.IDENTITY and fields for id, name, email and password.

4. User Service

The User Service is responsible for user information. The supplied screenshots show UserController.java, User.java and UserRepository.java in the service structure.

User entity

- User is marked with @Entity.
- The id field is marked with @Id.
- The id uses @GeneratedValue(strategy = GenerationType.IDENTITY).
- The visible fields are id, name, email and password.
- The class contains a default constructor, parameterized constructor, getters and setters.

Typical REST operations

Method	Endpoint	Purpose
POST	/users	Create a user
GET	/users	Get all users
GET	/users/{id}	Get a user by ID
PUT	/users/{id}	Update a user
DELETE	/users/{id}	Delete a user

Important testing note

One supplied Postman screenshot shows GET http://localhost:8083/users/1 returning 404 Not Found. A 404 at this endpoint normally means the requested route is not mapped in the running application, the wrong service/port is being called, or the controller path differs from the URL. Before final submission, verify the UserController mapping and the User Service port.

5. Product Service

The Product Service manages product information. The supplied project screenshots show `ProductRepository.java`, `ProductController.java` and `ProductServiceApplication.java`.

Product API operations

Method	Endpoint	Purpose
POST	/products	Create a product
GET	/products	Get all products
GET	/products/{id}	Get a product by ID
PUT	/products/{id}	Update a product
DELETE	/products/{id}	Delete a product

Postman evidence visible in the supplied screenshots

- POST /products was tested.
- GET /products was tested.
- GET /products/4 and PUT /products/4 were tested.
- DELETE /products/4 was tested.
- A successful GET response visible in the earlier project work contained a product with ID 1, name 'laptop', quantity 2 and price 50000.

This demonstrates the CRUD lifecycle: create → read → update → delete.

6. Order Service and Inter-Service Communication

The supplied OrderController.java screenshot shows an Order Service controller using Spring's RestTemplate. The controller contains a health-style endpoint and an endpoint that calls another service.

Visible controller design

Endpoint	Purpose
/orders	Checks that the Order Service is running and returns a service status message.
/orders/user/{id}	Calls a user-related service using RestTemplate and returns the response.

Conceptual inter-service request

Postman → Order Service → RestTemplate → User Service → User response → Order Service → Postman

Troubleshooting observed in supplied screenshots

- GET http://localhost:8083/orders/user/1 returned HTTP 500 Internal Server Error in one screenshot.
- This indicates the Order Service received the request but an exception occurred while processing it.
- The first checks should be: confirm the User Service is running, confirm its port, confirm the User Service endpoint path, and verify the RestTemplate URL used in OrderController.
- Test the User endpoint directly in Postman first. Only after it returns successfully should the Order → User call be tested.

Recommended testing order

1) Start User Service → 2) test User API directly → 3) start Order Service → 4) test /orders → 5) test /orders/user/{id}.

7. Database, Validation and Exception Handling

Database configuration

Each microservice should have independent persistence configuration. The database URL, username and password belong in that service's application.properties. Avoid sharing entity classes or repositories between services.

Example configuration pattern

```
spring.datasource.url=jdbc:mysql://localhost:3306/  
spring.datasource.username=  
spring.datasource.password=  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true
```

Validation

- Validate required fields before saving data.
- Use annotations such as @NotBlank, @Email, @Positive or @Min where appropriate.
- Return HTTP 400 Bad Request when input is invalid.

Exception handling

- Return HTTP 404 Not Found when a requested entity does not exist.
- Return HTTP 400 Bad Request for invalid input.
- Return HTTP 500 Internal Server Error only for unexpected server-side failures.
- A @ControllerAdvice-based global exception handler is recommended for consistent JSON error responses.

Security note

For a learning project, the User entity may contain a password field as shown in the supplied screenshot. For a real application, passwords must never be stored as plain text; they should be securely hashed and protected.

8. API Testing with Postman

The task explicitly requires testing all APIs using Postman. Use the following sequence so failures are easy to isolate.

Order	Test	Expected result
1	Start User Service	Application starts without errors.
2	POST /users	HTTP 200/201 with created user.
3	GET /users/{id}	HTTP 200 with user data.
4	PUT /users/{id}	HTTP 200 with updated data.
5	DELETE /users/{id}	HTTP 200/204.
6	POST /products	HTTP 200/201 with created product.
7	GET /products	HTTP 200 with product list.
8	PUT /products/{id}	HTTP 200 with updated product.
9	DELETE /products/{id}	HTTP 200/204.
10	Start Order Service	Application starts without errors.
11	GET /orders	HTTP 200 with service status.
12	GET /orders/user/{id}	HTTP 200 after User Service communication is verified.

Troubleshooting checklist

- 404 Not Found → check controller @RequestMapping/@GetMapping path and port.
- 500 Internal Server Error → inspect the Order Service terminal for the root exception.
- Connection refused → confirm the target Spring Boot service is running.
- Empty database response → verify database connection and the requested ID.
- Wrong service response → verify the URL/port used by RestTemplate.

Evidence

The supplied screenshots show Postman history for User, Product and Order requests, including CRUD requests for Product and service-to-service requests for Order. These screenshots can be retained as evidence alongside the final repository.

9. README and Git/GitHub Delivery

README should contain

- Project title and objective.
- Architecture diagram.
- List of services and responsibilities.
- Technology stack.
- How to run each service.
- Database configuration instructions.
- API endpoint list.
- Postman testing instructions.
- Inter-service communication explanation.
- Known limitations/troubleshooting notes.

Git delivery evidence

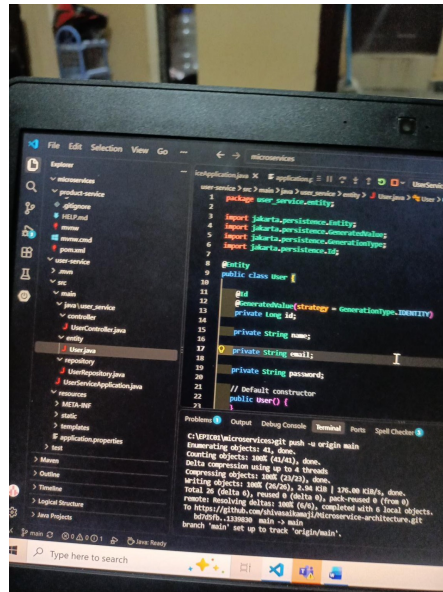
The supplied terminal screenshot shows a successful push to the main branch with the message "branch 'main' set up to track 'origin/main'" and a GitHub repository named **Microservice-architecture**. The GitHub screenshot also shows the service folders **order-service**, **product-service** and **user-service**.

Final verification checklist

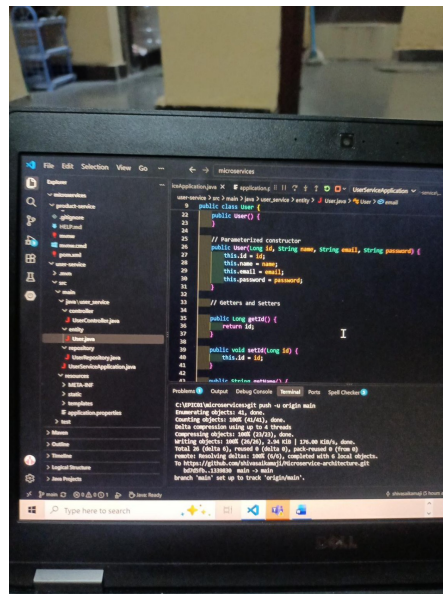
Requirement	Status to verify before submission
Spring Boot services	User, Product and Order services start successfully.
Database	Each service connects to its intended database/schema.
Entities	JPA entities are present and mapped correctly.
Repositories	Spring Data repositories work.
Services	Business logic is separated from controllers.
REST controllers	Required endpoints return expected HTTP responses.
Inter-service communication	Order → User request works end-to-end.
Validation	Invalid request data is rejected correctly.
Exception handling	404/400/500 responses are handled consistently.
Postman	All required APIs have successful test evidence.
Architecture diagram	Included in README/documentation.
README	Setup and API instructions included.
Git/GitHub	Latest working source code is pushed.

10. Appendix — Supplied Project Evidence

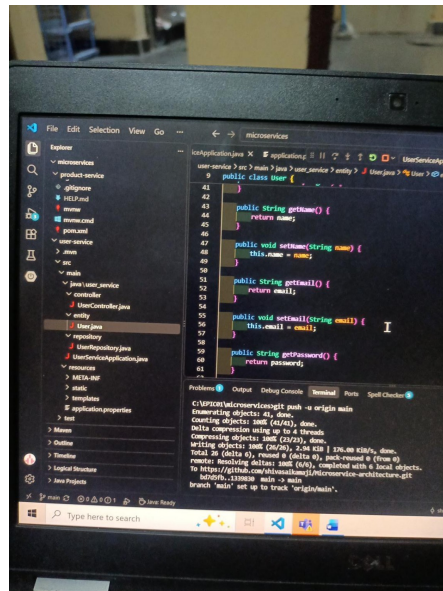
The following screenshots were supplied during the task and are included as implementation evidence.



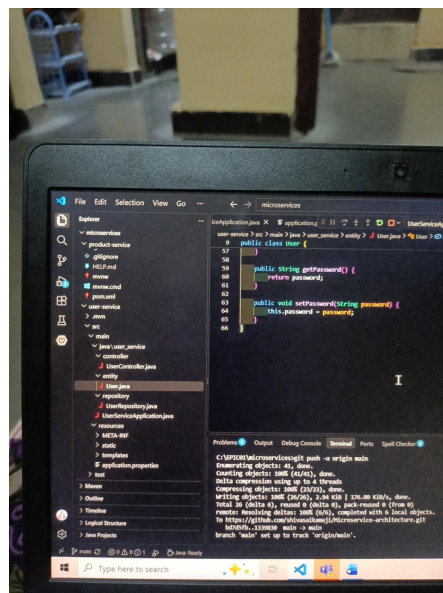
Postman: Order Service request /orders/user/1 showing HTTP 500 Internal Server Error.



Postman: User Service request /users/1 showing HTTP 404 Not Found.



VS Code: OrderController.java and terminal showing Git push output.



VS Code: User.java entity and Git push output.

Submission note: Before submitting, make sure the latest code is pushed to GitHub and that the User and Order endpoints shown with 404/500 in the supplied evidence have been re-tested successfully. The documentation intentionally records those observed responses rather than claiming a successful result that is not visible in the evidence.